A Comparative Study of Neural-Network Wave Functions for Trapped Bosonic Systems

Davide Cacopardi, Oskar Fausko, Teresa Macarulla

April 2026

Abstract

We study trapped bosonic systems using three neural-network-based approaches: a Restricted Boltzmann Machine (RBM), a feedforward Neural Network (NN), and a Physics-Informed Neural Network (PINN). The systems are confined in isotropic and anisotropic harmonic traps, with and without a repulsive two-body interaction. For the non-interacting cases, the numerical results are benchmarked against analytical harmonic-oscillator energies, while the interacting cases are compared through variational energy estimates and statistical uncertainties. The RBM implementation accurately reproduces the non-interacting ground-state energies and provides stable variational estimates for moderate interacting systems, although its optimization becomes more sensitive to hyperparameters and system size. The feedforward NN improves the interacting energies relative to the RBM in the tested cases, but requires a more elaborate training procedure involving pre-training and adiabatic introduction of the interaction. The PINN reproduces the non-interacting analytical results with very high accuracy and gives the lowest energies for the interacting anisotropic systems where it is applied. However, its performance depends strongly on the sampling strategy, especially for larger interacting systems. Overall, the results show that neural-network quantum states can successfully approximate trapped-boson ground states.

Introduction

In this project we use different numerical methods to study a bosonic gas confined in harmonic traps. A central difficulty in quantum many-body physics is that the wave function is a high-dimensional object whose complexity grows rapidly with the number of particles. Even when the Hamiltonian is simple, finding a good trial wave function may be difficult, and in more complicated cases an accurate analytical ansatz may simply be unknown. This motivates the use of alternative numerical approaches beyond standard Variational Monte Carlo (VMC), where the accuracy depends on the choice of a suitable trial wave function. By introducing neural-network quantum states, instead of prescribing the detailed structure of the wave function by hand, we can rather use trainable parameters to represent the wave function [1].

This work builds on previous projects in which bosons confined in the same harmonic trap were studied within the standard VMC framework using a manually constructed trial wave function [2, 3, 4]. Here, we extend that approach by considering the same harmonic confinement but with Coulomb interparticle interactions. This allows us to compare the performance of multiple numerical methods and assess how different representations of the wave function affect the results. In particular, we replace the analytical trial wave function with a Restricted Boltzmann Machine (RBM), and further consider a feedforward Neural Network (NN) with increased flexibility. These two approaches are based on the work of Carleo and Troyer, and Saito in [1, 5] respectively. Finally, we employ a Physics-Informed Neural Network (PINN) to approximate the wave function, and compare the different methods with standard VMC results and relevant benchmarks. In this report, we first present the necessary theoretical background and describe the implementation of the different methods. We then present the results and conclude with a comparison of their accuracy and computational efficiency.

The complete code used to build the system, run the simulations, post-process the results, and generate the figures is available in the GitHub repository[1].

Fundamentals

Bosonic system and Hamiltonian

We consider a system of identical bosons confined in a harmonic trap. The general structure of the Hamiltonian is the same as in standard variational treatments of trapped quantum particles: a one-body contribution containing the kinetic energy and the external confinement, plus a two-body interaction term. In natural units, this can be written as

$$\hat{H} = \sum_{i=1}^{N} \left(-\frac{1}{2}\nabla_i^2 + \frac{1}{2}\omega^2 r_i^2 \right) + \sum_{i<j} \frac{1}{r_{ij}}. \qquad (1)$$

where N is the number of particles, $r_i = |\mathbf{r}_i|$, and $r_{ij} = |\mathbf{r}_i - \mathbf{r}_j|$ [6].

The Hamiltonian itself does not depend on whether the particles are fermions or bosons; the difference enters through the symmetry of the many-body wave

[1] https://github.com/OzzyTheFuzzy/FYS4411-Project-2

function. Since in the present report we focus on a bosonic system, the relevant spatial wave function must be symmetric under exchange of any pair of particles,

$$\Psi(\mathbf{r}_1, \mathbf{r}_2, \ldots, \mathbf{r}_P) = \Psi(\mathbf{r}_{\pi(1)}, \mathbf{r}_{\pi(2)}, \ldots, \mathbf{r}_{\pi(P)}),$$

for any permutation π.

It is useful to separate the Hamiltonian into a non-interacting part and an interaction part,

$$\hat{H} = \hat{H}_0 + \hat{H}_1, \tag{2}$$

with

$$\hat{H}_0 = \sum_{i=1}^{P} \left(-\frac{1}{2}\nabla_i^2 + \frac{1}{2}\omega^2 r_i^2 \right),$$

or for the elliptic case

$$\hat{H}_0 = \sum_{i=1}^{N} \left[-\frac{1}{2}\nabla_i^2 + \frac{1}{2}(\omega_x^2 r_{xi}^2 + \omega_y^2 r_{yi}^2 + \omega_z^2 r_{zi}^2) \right], \tag{3}$$

and

$$\hat{H}_1 = \sum_{i<j} \frac{1}{r_{ij}}. \tag{4}$$

The term $\hat{H}_0$ defines the non-interacting harmonic-oscillator benchmark, while $\hat{H}_1$ introduces correlations between particles.

For a single particle in a two-dimensional isotropic harmonic oscillator, the eigenfunctions are [6]

$$\phi_{x,y}(x,y) = A\, H_{n_x}(\sqrt{\omega}\, x)\, H_{n_y}(\sqrt{\omega}\, y) \exp\Big[-\frac{\omega}{2}(x^2 + y^2) \Big], \tag{5}$$

where H_{n_x} and H_{n_y} are Hermite polynomials and A is a normalization constant. For the lowest state, $n_x = n_y = 0$, the one-particle energy is $\epsilon_{0,0} = \omega$. Therefore, in the non-interacting case the ground-state energy of two bosons in the trap is simply $E_0 = 2\omega$.

The corresponding unperturbed two-body bosonic ground-state wave function is the symmetric product of two lowest-lying oscillator orbitals,

$$\Phi(\mathbf{r}_1, \mathbf{r}_2) = C \exp\Big[-\frac{\omega}{2}(r_1^2 + r_2^2) \Big], \tag{6}$$

with C a normalization constant. This state provides the natural analytical benchmark before interactions are introduced. In the specific two-particle problem, setting $\omega = 1$, one has the exact non-interacting energy $E_0 = 2$, while the interacting benchmark gives $E = 3$ for the selected solvable case.

Within the variational framework, the central quantity is the local energy,

$$E_L(\mathbf{R}) = \frac{1}{\Psi(\mathbf{R})} \hat{H} \Psi(\mathbf{R}), \tag{7}$$

where $\mathbf{R} = (\mathbf{r}_1, \ldots, \mathbf{r}_P)$ and Ψ is the trial wave function. The exact ground-state energy, considering a non-interacting isotropic trap, is

$$E_{exact} = \frac{ND}{2}, \tag{8}$$

for N bosons in D spatial dimensions. For the non-interacting anisotropic trap, we have

$$E_{exact} = N\left(1 + \frac{\omega_z}{2}\right). \tag{9}$$

The main difference between the methods considered in this report is therefore not the Hamiltonian itself, but the way the trial state Ψ is represented.

Restricted Boltzmann Machine (RBM)

Just like wavefunctions, neural networks (NNs) are essentially functions taking a certain number of inputs and providing a number of outputs. The internal structure of a NN is divided into layers, which in turn are composed of several nodes. In principle, all nodes can communicate with one another and networks can be concatenated too. However, the more intricate the structure the harder it is to train it.

In order to simplify the process, we choose to work with RBMs. These consist of only one *hidden layer* and a *visible layer* (see Figure 1). Furthermore, nodes within each layer do not communicate.

1. The N_{in} nodes in the visible layer are represented by a function $\mathbf{x}$. This layer represents both what the RBM might be given as training input, and what we want it to be able to reconstruct.

2. The N_{hid} nodes in the hidden layer are represented by a function $\mathbf{h}$.

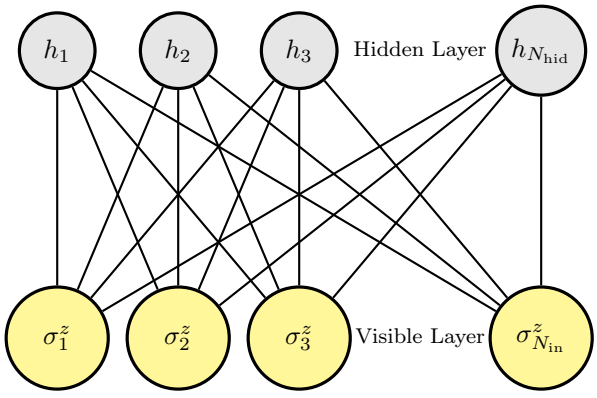

Figure 1: Restricted Boltzmann machine architecture with visible and hidden layers [1].

The goal of the hidden layer is to increase the model's expressive power. We encode complex interactions between visible variables by introducing hidden variables that interact with visible degrees of freedom.

Trial wave function as a NN

The RBM is described by a Boltzmann distribution

$$F_{rbm}(\mathbf{x}, \mathbf{h}) = \frac{1}{Z} \exp\Big(-\frac{1}{T_0} E(\mathbf{x}, \mathbf{h}) \Big), \tag{10}$$

where Z is the partition function

$$Z = \iint \mathrm{d}\mathbf{x}\mathrm{d}\mathbf{h} \exp\Big(-\frac{1}{T_0}E(\mathbf{x},\mathbf{h})\Big), \tag{11}$$

and we set $T_0 = 1$.

The function $E(\mathbf{x},\mathbf{h})$ gives the energy of a configuration $(\mathbf{x},\mathbf{h})$. The lower the energy of a configuration, the higher its probability. We choose the expression of such function to depend on a set of parameters. The learning process optimizes these parameters so that the energy is led to a minimum.

There are multiple variants of RBMs. The simplest is called *Binary-Binary RBM* and allows each node to take only one binary value (i.e., $x_i, h_j \in \{0,1\}$). The set of parameters $\boldsymbol{\theta}$ comprises:

1. a vector $\mathbf{a}$ of same length as $\mathbf{x}$, which represents the visible bias;
2. a vector $\mathbf{b}$ of same length as $\mathbf{h}$, which represents the hidden bias;
3. a matrix $W = \{w_{ij}\}_{j=1,\dots,N_{\mathrm{hid}}}^{i=1,\dots,N_{\mathrm{in}}}$, which represents the interaction weights;

and the energy function can be defined as

$$E(\mathbf{x},\mathbf{h}) = -\sum_{i=1}^{N_{\mathrm{in}}} a_i x_i - \sum_{j=1}^{N_{\mathrm{hid}}} b_j h_j - \sum_{i=1}^{N_{\mathrm{in}}}\sum_{j=1}^{N_{\mathrm{hid}}} x_i w_{ij} h_j. \tag{12}$$

Other typical models are *Gaussian-Binary RBMs* and *Gaussian-Gaussian RBMs*, which are more useful to model continuous data. Indeed, the *Gaussian-Binary RBMs* is the case we are interested in, and the energy function is thus defined as

$$E(\mathbf{x},\mathbf{h}) = \sum_{i=1}^{N_{\mathrm{in}}} \frac{(x_i - a_i)^2}{2\sigma_i^2} - \sum_{j=1}^{N_{\mathrm{hid}}} b_j h_j - \sum_{i=1}^{N_{\mathrm{in}}}\sum_{j=1}^{N_{\mathrm{hid}}} \frac{x_i w_{ij} h_j}{\sigma_i^2}. \tag{13}$$

To use the RBM as a variational ansatz in quantum mechanics, we need the probability distribution defined by the network only in terms of the visible variables. This is obtained by marginalizing over the hidden layer,

$$F_{\mathrm{rbm}}(\mathbf{x}) = \sum_{\mathbf{h}} F_{\mathrm{rbm}}(\mathbf{x},\mathbf{h}) = \frac{1}{Z}\sum_{\mathbf{h}} e^{-E(\mathbf{x},\mathbf{h})}. \tag{14}$$

For the Gaussian–binary RBM with $T_0 = 1$ and, for simplicity, a common width $\sigma_i = \sigma$, this sum can be carried out analytically and yields

$$\begin{aligned} F_{\mathrm{rbm}}(\mathbf{x}) = \frac{1}{Z}\exp\left[-\sum_{i=1}^{N_{\mathrm{in}}}\frac{(x_i-a_i)^2}{2\sigma^2}\right] \\ \times \prod_{j=1}^{N_{\mathrm{hid}}}\left(1+\exp\left[b_j + \sum_{i=1}^{N_{\mathrm{in}}}\frac{x_i w_{ij}}{\sigma^2}\right]\right). \end{aligned} \tag{15}$$

This marginal distribution is then identified with the variational wave function,

$$\Psi_{\mathrm{RBM}}(\mathbf{x}) \equiv F_{\mathrm{rbm}}(\mathbf{x}), \tag{16}$$

up to an overall normalization factor. In the present bosonic problem, the visible variables $\mathbf{x}$ correspond to the particle coordinates, so that the number of visible nodes is fixed by the number of particles and spatial dimensions, $M = P \cdot D$. The hidden variables do not correspond to physical particles; instead, they provide additional flexibility that allows the ansatz to encode correlations between the physical degrees of freedom.

The importance of Eq. (16) is that it turns the RBM into a genuine trial state for VMC. Once Ψ_{RBM} has been defined, it can be inserted into the same variational machinery used in standard Monte Carlo calculations. In particular, the local energy is evaluated as

$$E_L(\mathbf{x}) = \frac{1}{\Psi_{\mathrm{RBM}}(\mathbf{x})}\hat{H}\Psi_{\mathrm{RBM}}(\mathbf{x}), \tag{17}$$

and the RBM parameters $\{a_i, b_j, w_{ij}\}$ are optimized by minimizing the expectation value of the Hamiltonian.

Analytical expressions

To use the Restricted Boltzmann Machine as a variational ansatz within VMC, we need explicit analytical expressions for the local energy and for the derivatives of the wave function with respect to the variational parameters. These expressions are the basic ingredients required to optimize the neural-network quantum state.

For the *Gaussian–binary* RBM, the wave function obtained after marginalizing over the hidden variables is described by Eq. (15). Since an overall normalization constant cancels in Monte Carlo expectation values, we can omit the partition function Z.

It is convenient to define

$$\theta_j(\mathbf{X}) = b_j + \sum_{i=1}^{N_{\mathrm{in}}}\frac{X_i w_{ij}}{\sigma^2}, \tag{18}$$

so that

$$\ln\Psi(\mathbf{X}) = -\sum_{i=1}^{N_{\mathrm{in}}}\frac{(X_i-a_i)^2}{2\sigma^2} + \sum_{j=1}^{N_{\mathrm{hid}}}\ln\Big(1+e^{\theta_j(\mathbf{X})}\Big). \tag{19}$$

The Hamiltonian of the bosonic system can be written in the compact form of Eq. (1).

The local energy is defined in Eq. (17). Using the identity

$$\frac{1}{\Psi}\frac{\partial^2\Psi}{\partial X_i^2} = \frac{\partial^2\ln\Psi}{\partial X_i^2} + \left(\frac{\partial\ln\Psi}{\partial X_i}\right)^2, \tag{20}$$

the local energy can be written as

$$E_L(\mathbf{X}) = -\frac{1}{2}\sum_{i=1}^{N_{\mathrm{in}}}\left[\frac{\partial^2\ln\Psi}{\partial X_i^2} + \left(\frac{\partial\ln\Psi}{\partial X_i}\right)^2\right] + V(\mathbf{X}), \tag{21}$$

where $V(\mathbf{X})$ accounts for the second part of the sum in Eq. (3) and Eq.(4).

From Eq. (19), the first derivatives are

$$\frac{\partial \ln \Psi}{\partial X_i} = -\frac{X_i - a_i}{\sigma^2} + \sum_{j=1}^{N_{\text{hid}}} \frac{w_{ij}}{\sigma^2} \frac{1}{1+e^{-\theta_j}}, \tag{22}$$

and the second derivatives are

$$\frac{\partial^2 \ln \Psi}{\partial X_i^2} = -\frac{1}{\sigma^2} + \sum_{j=1}^{N_{\text{hid}}} \frac{w_{ij}^2}{\sigma^4} \frac{e^{-\theta_j}}{\left(1+e^{-\theta_j}\right)^2}. \tag{23}$$

Substituting Eqs. (22) and (23) into Eq. (21) gives the explicit RBM expression for the local energy.

To optimize the variational parameters, we also need the logarithmic derivatives of the wave function with respect to the parameters

$$\alpha_k \in \{a_1, \ldots, a_M, b_1, \ldots, b_N, w_{11}, \ldots, w_{MN}\}. \tag{24}$$

These derivatives are given by;

$$\frac{1}{\Psi}\frac{\partial \Psi}{\partial a_i} = \frac{X_i - a_i}{\sigma^2}, \tag{25}$$

$$\frac{1}{\Psi}\frac{\partial \Psi}{\partial b_j} = \frac{1}{1+e^{-\theta_j}}, \tag{26}$$

and

$$\frac{1}{\Psi}\frac{\partial \Psi}{\partial w_{ij}} = \frac{X_i}{\sigma^2}\frac{1}{1+e^{-\theta_j}}. \tag{27}$$

Defining

$$O_k(\mathbf{X}) \equiv \frac{1}{\Psi(\mathbf{X})}\frac{\partial \Psi(\mathbf{X})}{\partial \alpha_k}, \tag{28}$$

the gradient of the variational energy takes the standard VMC form

$$G_k = \frac{\partial \langle E_L \rangle}{\partial \alpha_k} = 2\left(\langle E_L O_k \rangle - \langle E_L \rangle \langle O_k \rangle\right). \tag{29}$$

This quantity is the basic ingredient entering optimization schemes.

In summary, Eqs. (21)–(29) provide the analytical formulas needed to implement the RBM variational ansatz in the Monte Carlo framework. The practical workflow is then the same as in ordinary VMC: sample configurations from $|\Psi(\mathbf{X})|^2$, evaluate the local energy, estimate the gradients, and update the parameters until convergence.

Feedforward Artificial Neural Network (Feedforward NN)

A similar approach is the one employed in the work of H. Saito [5], which utilizes a feedforward artificial NN instead of the RBM. Such NN is structured as follows.

We consider a visible layer of N_{in} input nodes $u_1^{\text{in}}, ..., u_{N_{\text{in}}}^{\text{in}}$, connected to a hidden layer of N_{hid} nodes $u_1^{\text{hid}}, ..., u_{N_{\text{hid}}}^{\text{hid}}$ according to the relation

$$u_i^{\text{hid}} = \sum_{j=1}^{N_{\text{in}}} W_{ji}^{(1)} u_j^{\text{in}} + b_i^{(1)}, \tag{30}$$

where the weights $W_{ji}^{(1)}$ and biases $b_i^{(1)}$ are real numbers. The output of the network is a single value given by

$$u^{\text{out}} = \sum_{i=1}^{N_{\text{hid}}} W_i^{(2)} f(u_i^{\text{hid}}), \tag{31}$$

where the weights $W_i^{(2)}$ are real numbers and f is an activation function.
Summing up, the NN's behavior depends on a set of $N_{hid}(N_{in}+2)$ parameters: $(W_{ji}^{(1)}, b_i^{(1)}, W_i^{(2)})$.

We can choose to input values other than the raw particle coordinates into the NN. That is, we input $\boldsymbol{\xi}(\mathbf{X}) = (\xi_1(\mathbf{X}), ..., \xi_{N_{\text{in}}}(\mathbf{X}))$, where $\mathbf{X}$ is the usual vector of all the particles' position vectors.
Finally, the many-body wavefunction is given by the output unit as[2] $\psi(\mathbf{X}) = \exp(u^{\text{out}})$.

Differently from other studied wavefunction models, this network needs a two-step training process in order to behave as the ground state's wavefunction correctly. It is important to choose initial values of the network parameters because if one uses random numbers for this purpose and starts the update procedure, the network state strongly fluctuates and the calculation is known to break down in most cases.

Pre-training

During this first training (or pre-training) process, the parameters are optimized in order to make the NN behave like a known wavefunction φ, generally chosen to be the analytical non-interacting wavefunction of the system. More specifically, we want to maximize the overlap integral K between φ and the wavefunction obtained from our NN $\psi = \exp(u^{\text{out}})$.

$$K = \frac{\left[\int \varphi(\mathbf{X})\psi(\mathbf{X})\,\mathrm{d}\mathbf{X}\right]^2}{\int \varphi^2(\mathbf{X})\,\mathrm{d}\mathbf{X} \int \psi^2(\mathbf{X})\,\mathrm{d}\mathbf{X}} \tag{32}$$

This is accomplished by providing the *Adam scheme* with the derivatives $\frac{\partial K}{\partial W}$ with respect to the network parameters.

Operatively, one can evaluate this expression using the same Monte Carlo integrations implemented for the energy calculations, where the expectation value of a generic operator $\hat{\mathcal{C}}$ acting on a wavefunction is given by

$$\begin{aligned} \langle \hat{\mathcal{C}} \rangle &= \frac{\int \psi(\mathbf{X}) \hat{\mathcal{C}} \psi(\mathbf{X}) \mathrm{d}\mathbf{X}}{\int \psi(\mathbf{X})^2 \mathrm{d}\mathbf{X}} \\ &\simeq \frac{1}{N_{\text{sample}}} \sum_{i=1}^{N_{\text{sample}}} \psi^{-1}(\mathbf{X}_i) \mathcal{C} \psi(\mathbf{X}_i) \\ &= \langle \psi^{-1} \hat{\mathcal{C}} \psi \rangle_\psi. \end{aligned} \tag{33}$$

Indeed, by defining $A = \varphi/\psi$, it is possible to rearrange Eq. (32) to obtain $K \simeq \langle A \rangle_\psi^2 / \langle A^2 \rangle_\psi$. Conversely, we can define $B = \psi/\varphi$ and obtain $K \simeq$

[2] Note that this is not the final wavefunction form, as explained in detail in the following implementation subsection, "*Gaussian envelope*".

$\langle B\rangle_\varphi^2/\langle B^2\rangle_\varphi$, sampling from the probability distribution shaped by the chosen known wavefunction φ.
In our implementation, we chose this second expression to evaluate K and its derivatives

$$\frac{\partial K}{\partial W} = 2K\left(\frac{\langle BO_W\rangle_\varphi}{\langle B\rangle_\varphi} - \frac{\langle B^2O_W\rangle_\varphi}{\langle B^2\rangle_\varphi}\right) \tag{34}$$

in order to stabilize the Metropolis algorithm[3].

Training

After the pre-training step, the energy of the system can be optimized following similar procedures as the ones employed in Project 1 and with RBMs. The optimization method of choice is—once again—the *Adam scheme*, fed with the derivatives of the energy with respect to the variational parameters (see Eq. (29)). The only substantial difference is that the interaction potential is introduced adiabatically, over the first N_{adiab} training steps. Mathematically, this means that the Hamiltonian of the system after n steps is:

$$\hat{H}(n) = \begin{cases} \hat{H}_0 + (n/N_{\text{adiab}})\hat{H}_1 & \text{if } n < N_{\text{adiab}} \\ \hat{H}_0 + \hat{H}_1 & \text{if } n \geq N_{\text{adiab}} \end{cases}. \tag{35}$$

This prescription can prevent the calculation from breaking down due to the sudden introduction of an infinite potential and avoid erratic behaviors, more likely to lead into non-global minima.

Physics-Informed Neural Networks (PINNs)

For a system of identical bosons we need a method to ensure that the PINN model has permutation invariance. Meaning that our model satisfies

$$\mathcal{N}(\mathbf{x}_1,\ldots,\mathbf{x}_N) = \mathcal{N}(\mathbf{x}_N,\ldots,\mathbf{x}_1) \tag{36}$$

The method we choose to satisfy this is called deep sets.

Deep sets and PINN as wavefunction

The deep sets ansatz yields

$$f(\mathbf{X}) = \rho\left(\sum_{i=1}^{N}\phi(\mathbf{x}_i)\right), \tag{37}$$

where ϕ is a NN acting on the single-particle coordinates $\mathbf{x}_i$ [7].

$$f(\mathbf{x}_1,\ldots,\mathbf{x}_N) = f(\mathbf{x}_{\pi(1)},\ldots,\mathbf{x}_{\pi(N)}). \tag{38}$$

Using one NN, η, that takes the pairwise distances $r_{ij} = |\mathbf{x_i} - \mathbf{x_j}|$ and another NN, ϕ, that takes in all the single particle positions, we can combine the outcome of these two networks and feed into a final network ρ_θ. In this way, the model can take into account both the inter-particle correlations and the absolute distances from the center of the trap for all the particles

$$\rho_\theta(\mathbf{X}) = \rho_\theta\left(\sum_{i=1}^{N}\phi_\theta(\mathbf{x}_i), \sum_{i<j}\eta_\theta(r_{ij})\right). \tag{39}$$

To help the model capture the decay of the wavefunction, we add a logarithm of a gaussian term to the model

$$g(\mathbf{X}) = \left(-\sum_{i=1}^{N}\left[\alpha\left(x_i^2+y_i^2\right)+\beta z_i^2\right]\right), \tag{40}$$

where we set $\alpha = 0.5$ and $\beta = \omega_z$, where $\omega_z = 1.0$ for the isotropic trap and $\omega_z = 2.82843$ for the anisotropic trap. To include inter-particle correlations, we multiply the wavefunction by a Padé-Jastrow factor, $J(\mathbf{X})$, and include a Coulomb interaction term, $V_{\text{C}}(\mathbf{X})$, in the local energy, scaled with the parameter a. The Coulomb interaction term is given by

$$V_{\text{C}}(\mathbf{X}) = \begin{cases} 0, & a = 0.0, \\ \displaystyle\sum_{i<j}\frac{a}{r_{ij}}, & a = 1.0, \end{cases} \qquad r_{ij} = |\mathbf{x}_i - \mathbf{x}_j|. \tag{41}$$

The parameter a corresponds to the strength of the Coulomb interaction term and is $a = 0.0$ for the non-interacting case and $a = 1.0$ for the interacting case. This implementation is motivated by the cusp-condition structure commonly used in Coulomb systems, where singular contributions from the kinetic and interaction terms cancel at small distances [8]. The Jastrow factor suppresses configurations where particles approach each other too closely, improving the numerical stability of the local energy at short particle-particle distances. For improved numerical stability during training, we work with the logarithm of the wave function,

$$u_\theta(\mathbf{X}) = \log\psi_\theta(\mathbf{X}), \tag{42}$$

where

$$u_\theta(\mathbf{X}) = -\sum_{i=1}^{N}\left[\alpha(x_i^2+y_i^2)+\beta z_i^2\right] + \rho_\theta(\mathbf{X}) + J(\mathbf{X}). \tag{43}$$

The Padé-Jastrow factor is a smoother version of the standard hard core Jastrow factor used in project 1. The Padé-Jastrow factor is given by

$$J(\mathbf{X}) = \begin{cases} 0, & a = 0.0, \\ \displaystyle\sum_{i<j}\frac{a_J r_{ij}}{1+\beta_J r_{ij}}, & a = 1.0, \end{cases} \tag{44}$$

where $r_{ij} = |\mathbf{x}_i - \mathbf{x}_j|$, a_J and β_J are Jastrow parameters which can be tuned during training or kept fixed [9]. In this work, we fix the parameters at $a_J = 1.0$ and $\beta_J = 0.5$. Compared to the standard hard-core Jastrow factor, the Padé–Jastrow form is smoother

[3] At first, we opted for $K \simeq \langle A\rangle_\psi^2/\langle A^2\rangle_\psi$ as suggested in [5], but the Metropolis random walkers were misguided towards regions far from the center by the randomly initialized NN, causing severe numerical stability issues.

and avoids singular or discontinuous behavior in the derivatives of the wave function, thereby improving numerical stability

$$j(r_{ij}) = \frac{a_J r_{ij}}{1+\beta_J r_{ij}}.$$

$$\begin{aligned} j''(r_{ij}) &= \frac{d^2}{dr_{ij}^2}\left(\frac{a_J r_{ij}}{1+\beta_J r_{ij}}\right) \\ &= \frac{d}{dr_{ij}}\left(\frac{a_J}{(1+\beta_J r_{ij})^2}\right) \\ &= -\frac{2a_J\beta_J}{(1+\beta_J r_{ij})^3}. \end{aligned}$$

Loss function

For training our model we use the PDE residual. The PDE residual comes from the stationary Schrödinger equation

$$\hat{H}\psi(\mathbf{X}) = E\psi(\mathbf{X}), \tag{45}$$

for the eigenstate ψ with energy E. In our case the Hamiltonian is given by Eq. (2) only switching out V_C given in (41) instead of (4).

Dividing Equation (45) by $\psi(\mathbf{X})$ and moving E to the left hand side gives

$$-\frac{1}{2}\sum_{i=1}^{N}\frac{\nabla_i^2\psi(\mathbf{X})}{\psi(\mathbf{X})} + V_{\mathrm{HO}}(\mathbf{X}) + V_{\mathrm{C}}(\mathbf{X}) - E = 0, \tag{46}$$

where $V_{\mathrm{HO}}(\mathbf{X})$ is equivalent to Eq. (3) and $V_{\mathrm{C}}(\mathbf{X})$ is given in Equation (41) Using the neural-network representation

$$\psi_\theta(\mathbf{X}) = e^{u_\theta(\mathbf{X})}, \tag{47}$$

one obtains

$$\frac{\nabla_i^2\psi_\theta}{\psi_\theta} = \nabla_i^2 u_\theta + |\nabla_i u_\theta|^2, \tag{48}$$

such that

$$E_K(\mathbf{X}) = -\frac{1}{2}\sum_{i=1}^{N}\left[\nabla_i^2 u_\theta(\mathbf{X}) + |\nabla_i u_\theta(\mathbf{X})|^2\right], \tag{49}$$

The PDE residual is therefore given by

$$R(\mathbf{X}) = E_K(\mathbf{X}) + V_{\mathrm{HO}}(\mathbf{X}) + V_{\mathrm{C}}(\mathbf{X}) - E_\theta, \tag{50}$$

where E_θ is a pre-calculated estimate of the energy using the analytical expression or the RBM estimate. The PDE loss is therefore given by

$$\mathcal{L}_{\mathrm{PDE}} = \frac{1}{B}\sum_{b=1}^{B} R(\mathbf{X}_b)^2, \tag{51}$$

where B is the number of configurations for a given training cycle and $R(\mathbf{X}_b)$ is the residual given by Equation (50). The model is trained by first evaluating the loss for a batch of sampled configurations. The model's parameters are then updated through backpropagation using automatic differentiation in order to minimize the loss function given in Equation (51).

The PINN energy, E_{PINN}, for a given system is estimated as

$$E_{\mathrm{PINN}} = \frac{1}{M}\sum_{m=1}^{M} E_L(\mathbf{X}_m), \tag{52}$$

where

$$E_L(\mathbf{X}_m) = E_K(\mathbf{X}_m) + V_{\mathrm{HO}}(\mathbf{X}_m) + V_{\mathrm{C}}(\mathbf{X}_m), \tag{53}$$

is the local energy for configuration $\mathbf{X}_m$. Since the neural network defines a variational trial wave function, the corresponding expectation value of the Hamiltonian satisfies the variational principle and therefore provides an upper bound to the ground-state energy. This interpretation assumes that the configurations $\mathbf{X}_m$ are sampled from the probability distribution associated with the trial wave function, as is the case in a VMC simulation

Implementation

RBM

The Restricted Boltzmann Machine was implemented as a variational wave function inside the VMC structure. For a system with N particles in D dimensions and N_{hid} hidden nodes, the trainable RBM parameters are the visible biases $a_{i,d}$, the hidden biases b_h, and the weights $W_{i,d,h}$. The visible layer therefore has dimension $N \times D$, the hidden-bias vector has dimension N_{hid}, and the weight tensor has dimension $N \times D \times N_{\mathrm{hid}}$.

At each Monte Carlo step, the local energy E_L and the logarithmic derivatives of the wave function with respect to the RBM parameters are evaluated. The energy gradient is then estimated using the standard VMC expression

$$G_k = 2\left(\langle E_L O_k\rangle - \langle E_L\rangle\langle O_k\rangle\right), \qquad O_k = \frac{1}{\Psi}\frac{\partial\Psi}{\partial\alpha_k}, \tag{54}$$

where α_k represents any of the parameters $a_{i,d}$, b_h, or $W_{i,d,h}$ (see equations (25)-(27)). In practice, this gives three gradient objects:

$$G_a \in \mathbb{R}^{N\times D}, \qquad G_b \in \mathbb{R}^{N_{\mathrm{hid}}}, \qquad G_W \in \mathbb{R}^{N\times D\times N_{\mathrm{hid}}}. \tag{55}$$

The RBM parameters were optimized using an Adam optimizer [10]. Adam introduces, for each parameter, two auxiliary quantities: a first-moment estimate m and a second-moment estimate v. These quantities are initialized to zero with the same shape as the corresponding parameter arrays. Thus, the implementation stores independent moment arrays for a, b, and W:

$$m_a, v_a \in \mathbb{R}^{N\times D},\ m_b, v_b \in \mathbb{R}^{N_{\mathrm{hid}}},\ m_W, v_W \in \mathbb{R}^{N\times D\times N_{\mathrm{hid}}}. \tag{56}$$

At optimization iteration t, the moment estimates are updated as

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)G_t, \tag{57}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) G_t^2, \tag{58}$$

where the square is applied element-wise. The parameters are then updated according to

$$\alpha_t = \alpha_{t-1} - \eta_t \frac{m_t}{\sqrt{v_t} + \epsilon}, \tag{59}$$

with the bias-corrected learning rate

$$\eta_t = \eta \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}. \tag{60}$$

Here η is the learning rate, while β_1, β_2, and ϵ are the usual Adam hyperparameters. The same update rule is applied separately to all components of a, b, and W.

After the Adam optimization, the final production runs were performed with the optimized RBM parameters. To improve the reliability of the estimates, we used independent parallel replicas. Each replica was initialized with a different random seed and evolved independently, while all replicas used the same optimized RBM parameters. The final reported energy was obtained by averaging the energies from the different replicas.

The output files also store a timing variable denoted by CPU-s. In the implementation, this quantity is measured inside each call to VMC cycle using a high-resolution clock. For the RBM calculations, the timer starts after the system has been initialized and after the equilibration steps have been completed, and it stops after the production Metropolis sampling has finished. Therefore, CPU-s measures the computational time in seconds spent in the production sampling stage of a given VMC run, including the evaluation of the local energy and the RBM parameter gradients. It does not include the initial construction of the system or the equilibration stage.

For parallel runs, each replica has its own value of CPU-s. The value represented in the results tables is the average over replicas. This should be distinguished from the total wall-clock time of the parallel calculation: if several replicas are run simultaneously, the average replica time measures the typical cost of one independent Markov chain, while the wall-clock time measures the real elapsed time of the parallel run.

Finally, the local-energy history was stored for each replica. This was used to perform a blocking analysis, which provides an uncertainty estimate that accounts for autocorrelation in the Markov chain. For further information about the blocking implementation see [2].

Feedforward NN

Similarly to the RBM, we implemented the artificial NN on top of the pre-existing VMC program designed for Project 1 [3]. The input-encoding function $\boldsymbol{\xi}$ was chosen to simply unpack the particles' coordinates[4]:

$$\begin{aligned}\boldsymbol{\xi}(\mathbf{X}) = (&x_{1,1}, x_{1,2}, ..., x_{1,D},\\ &x_{2,1}, x_{2,2}, ..., x_{2,D},\\ &...,\\ &x_{N,1}, x_{N,2}, ..., x_{N,D}).\end{aligned} \tag{61}$$

The C++ `libtorch` API was used to manage the forward and backward passes of the NN.
The following measures have been adopted in order to optimize the final implementation.

Gaussian envelope

The pre-training steps are tailored to maximize the overlap integral K between the network and the given wavefunction φ. This is achieved by sampling the particles' position $\mathbf{X}$ according to the $|\varphi|^2$ probability distribution. Although this choice improves the stability of the pre-training, the network's behavior on scarcely sampled regions (i.e.: farther from the peaks of $|\varphi(\mathbf{X})|^2$) isn't necessarily driven to zero and easily leads the Metropolis random walkers away from the relevant regions in the following energy optimization process, which samples directly from the probability distribution obtained by the NN.

The easiest workaround is to add a *Gaussian envelope* to the final node u^{out} when computing the final wavefunction:

$$\psi(\mathbf{X}) = \exp\big(u^{\text{out}} - \alpha|\mathbf{X}|^2\big), \tag{62}$$

where $|\mathbf{X}|^2 = \sum_{i=1}^{N}\sum_{j=1}^{D} x_{ij}^2$ and α is a fixed real parameter[5]. Throughout the final test runs, we kept $\alpha = 0.4$ fixed. However, it is left as one of the freely tunable parameters of the program (it goes by the name `helpDecay` in the code and in the configuration file).

Learning rate reduction on plateau

We utilized `libtorch`'s Adam optimization, initialized with the following setup method[6]: `AdamOptions(lr).betas({0.9,0.999}).eps(1e-6)`. After some tests, the optimal learning rate values `lr` seemed to be slightly greater than the recommended figures by a factor of ~ 50. Although Adam is an adaptive learning rate scheme in itself, we implemented a validation-loss-based learning-rate adjuster, in order to force a more precise convergence when a plateau is reached.
At each pre-training and training step (only after the adiabatic potential increase is

[4] We relied on the optimization algorithm to be, on average, symmetric under particle exchange, assuming this would sufficiently stabilize the NN to yield an approximately symmetric wavefunction. This symmetry could be explicitly enforced, for example, by choosing a different input vector $\boldsymbol{\xi}$.

[5] Note that the current implementation computes $|\mathbf{X}|^2 = \sum_{i=1}^{N_{\text{in}}} \xi_i^2(\mathbf{X})$, because of our choice (61). Different choices of ξ require adjustments to the evaluation of the *Gaussian envelope*.

[6] see `libtorch` reference.

over), the following method is called: `bool VMCOptimizer_NN::checkPlateau(double current_K, double& best_K, int& patience_counter, double min_improvement, bool increase)`. If the returned value is `true`, both the learning rate and the `min_improvement` parameter are divided by 10. If this happens during pre-training, these parameters are reset to their initial values before commencing the training process.

On a side note, we introduced a target threshold, `Adam_ktol`, for the overlap integral K, typically set to 0.99. This serves as a simple early-stopping mechanism to reduce computational cost: if the overlap reaches this value, further optimization is unnecessary and the program can proceed directly to the training phase.

The optimization process ends by printing the last set of parameters. These can be re-utilized to run a longer, dedicated Monte Carlo energy estimation. The local-energy history of this last run can be stored and a blocking analysis can be performed in the same way we did in Project 1 [3] and with RBMs. However, given the width of the noise observed even when approaching convergence, the final Monte Carlo run may easily occur when the Adam scheme is exploring an 'unfortunate' set of parameters, so we prefer to take the mean of the last $\sim 5\%$ of training steps as our final energy estimate for the purpose of this report's results.
Note that each training step changes the network parameters and equilibration steps are run in between every Monte Carlo, so we can consider the energy obtained from a set of parameters to be approximately statistically independent from the energy obtained from the subsequent set of parameters. Furthermore, an energy obtained by averaging over different wavefunction models may seem unphysical, but we have to keep in mind that the variational principle only provides upper-bounds to the true ground state energy. Working with neural networks, we can potentially shape their behavior very similarly to the true ground state wavefunction, but it's extremely unlikely for the internal structure of the neural network to truly fully match it. For this reason, averaging over final energies obtained from slightly different parameter choices may provide a more robust estimate of the ground-state energy.

In spite of these last arguments, we can run the one-body density routine as we did in Project 1 using the last training step's parameters for pictorial representation purposes.

PINNs

Sampling

To train a PINN, a set of particle configurations is required to evaluate the PDE residual and local energy during training. It is important to sample so that the network learns the physically relevant regions of configuration space. In Project 1, we observed that the particle distributions are approximately Gaussian, and we therefore sample particle configurations from Gaussian distributions. In Project 1, we observed that the bosonic gas expanded as interactions were introduced and the number of particles increased. This suggests that for larger particle ensembles, it may be beneficial to sample configurations from a broader distribution in order to better capture the physically relevant regions of configuration space. For the anisotropic case the sampling is harder. Since the total volume of the bosonic gas decreases, the sensitivity of the sampling increases. To counter this we introduce two different sampling methods. One of these methods is sampling from Gaussian distributions with different standard deviations in the $x-$,y-directions, compared to the z-direction. Since $\omega_z = 2.82843$, we use

$$\sigma_x = \sigma_y = \frac{1}{\sqrt{2}}, \qquad \sigma_z = \frac{1}{\sqrt{2\omega_z}}.$$

as standard deviations in the $x-$,$y-$, and z-directions, respectively. This sampling strategy serves as the primary method for generating particle configurations used in PINN training. The other sampling method is based on the estimate of the energy from a set of VMC sampled configurations. The configurations are generated by placing particles in three ellipsoidal shells. n_1 particles are placed close to the trap center, n_2 at an intermediate radius, and the remaining particles, n_3, in an outer shell. To generate these shells, particles are first sampled at a radial distance (s) in a random direction. The resulting coordinates are then transformed into an ellipsoidal shape by scaling the z-coordinate according to the trap anisotropy. Particle labels are randomly permuted to avoid encoding shell identity in the particle ordering. With appropriate choices of n_1, n_2, n_3 and the widths of the three shells, we can reproduce the same average harmonic and Coulomb potential energies, V_{HO} and V_{C}, defined in Equations (3) and (41), respectively, as those obtained from the VMC-sampled configurations. By matching these potential-energy contributions, the sampler may capture the spatial structure of the VMC-generated configurations more accurately than the standard Gaussian sampling method. Finally, when sampling for the interacting case, configurations where two particles are closer than $r_{\min}$ are rejected to avoid singular Coulomb contributions. This means that we calculate the distance between particles r_{ij} and reject if $r_{ij} < r_{\min}$.

Training

The energy of the model is estimated using Equation (52). We then compare our results with analytical values in the non-interacting case and with RBM results for the interacting case. Another benchmark we use is the variance of the local energy. If the model has reached an exact ground state, the variance should be is exactly zero, therefore, a small variance indicates that the model approaches a good approximation to

the ground state.

During training, we monitor the different contributions to the local energy, namely the kinetic energy, trap potential, and Coulomb interaction terms. These quantities are evaluated both on a fixed validation set and on configurations generated from an independent VMC simulation. In addition, we track the total training loss and validation loss throughout the training. Monitoring the individual energy contributions is useful for determining whether the initialized configurations produced a potential energy consistent with physically probable particle configurations. The kinetic energy is particularly sensitive to the derivatives of the wavefunction and therefore provides an indication of whether the learned wavefunction remains smooth and numerically stable during training. We plot the training and validation losses to determine whether the model is overfitting to the training data. If both the training and validation losses converge similarly, it suggests that the model generalizes beyond the training configurations and remains stable for unseen samples. To reduce overfitting, new particle configurations are generated at each training epoch. Furthermore, we compare the energy estimated from the validation set with the energy estimated from the VMC configurations in order to assess whether the training distribution is representative of physically relevant configurations. If there is a significant discrepancy between the two energy estimates we can adjust the width of the Gaussian that we sample from.

The energy eigenvalue for the PDE residual is $E_\theta = E_{RBM}$ for the interacting case, where E_{RBM} is the energy estimated by the Restricted Boltzmann Machine found in Table 5, presented later in the Results and Discussion. For the non-interacting case we use the analytical expression for the energy which is given by Equation (8) and (9). To prevent the model from converging to the trivial zero solution, we keep the eigenvalue E_θ constant in the non-interacting case. Since the RBM estimate of the energy comes with an uncertainty, allowing E_θ to remain a trainable parameter can help the system converge toward the true ground state in the interacting case. At the end of training, the loss curves are analyzed to determine whether the model has converged. If the loss continues to decrease, this may indicate that additional training epochs are needed.

Results and Discussion

In this section, we present the numerical results obtained with the three machine-learning approaches considered in this work: RBM, feedforward NN, and PINNs. We first study non-interacting systems, where analytical reference energies are available and the methods can be directly benchmarked. We then introduce the repulsive two-body interaction and compare the resulting variational energies, optimization behavior and statistical uncertainties. Throughout the discussion, we distinguish between the isotropic harmonic trap, where all oscillator frequencies are equal, and the anisotropic harmonic trap, where the confinement is stronger in one spatial direction.

RBM

Non-interacting isotropic harmonic trap

We first consider the non-interacting bosonic system in an isotropic harmonic trap. This case provides a direct benchmark because the exact ground-state energy is given by Eq. (8).

The results are shown in Table 1. In the following tables, E_{mean} denotes the numerical energy estimate obtained from the simulation, while E_{exact} denotes the analytical reference value when it is known. Both brute-force Metropolis and importance sampling reproduce the analytical energies for all tested systems. The agreement remains good when the number of visible degrees of freedom increases from $(N, D) = (1, 1)$ to $(10, 3)$, showing that the RBM ansatz correctly represents the Gaussian ground state of the non-interacting harmonic oscillator.

The main difference between the two samplers is seen in the acceptance ratio, in the autocorrelation and in the CPU-s counter. Importance sampling gives acceptance ratios close to unity because the proposal step is guided by the quantum force. However, the corresponding lag-1 correlations (correlation between consecutive samples) are also larger, meaning that consecutive local-energy samples are more strongly correlated. For this reason, the blocked standard error is the more reliable uncertainty estimate. Moreover, Importance sampling seems to consume more computational time.

Non-interacting anisotropic harmonic trap

We next consider the non-interacting bosonic system in an anisotropic trap with $\omega_x = \omega_y = 1$ and $\omega_z = 2.82843$. For this Hamiltonian, the exact ground-state energy in the anisotropic three-dimensional case is given by Eq. (9).

The results in Table 2 show that the RBM calculation reproduces the analytical energies for $N = 10$, 50, and 100 in three dimensions.

This benchmark is important because the trap is no longer isotropic. The remaining deviations from the analytical energies increase with N, but remain small compared with the total energy. As in the isotropic case, the lag-1 correlations are large for importance sampling, especially for the largest system, so the blocked uncertainties should be preferred over the naive standard errors.

Interacting isotropic harmonic trap

The interacting isotropic case was tested for the two-particle benchmark $(N, D) = (2, 2)$. For this system, the reference energy is $E = 3$. Table 3 shows the

Sampler	(N,D)	E_{mean}	E_{exact}	Acceptance	$SE^{\max}_{\text{naiv}}$	$SE^{\max}_{\text{block}}$	Lag-1 correlation	CPU-s
Brute force	(1,1)	0.499999	0.5	0.780	3×10^{-7}	1×10^{-6}	~ 0.857	2.691
Metropolis	(1,3)	1.5	1.5	0.591	7×10^{-7}	3×10^{-6}	~ 0.887	3.890
$sl = 0.8$	(10,3)	15	15	0.591	3×10^{-6}	4×10^{-5}	~ 0.988	24.190
Importance	(1,1)	0.500002	0.5	0.999	5×10^{-7}	5×10^{-6}	~ 0.980	3.498
sampling	(1,3)	1.499999	1.5	0.999	8×10^{-7}	7×10^{-6}	~ 0.980	5.184
$dt = 0.02$	(10,3)	14.9998	15	0.999	7×10^{-6}	2×10^{-4}	~ 0.998	36.575

Table 1: RBM benchmark for the non-interacting isotropic harmonic trap using $N_{\text{hid}} = 4$ hidden nodes. The exact reference energy is $E_{exact} = ND/2$. The columns $SE^{\max}_{\text{naiv}}$ and $SE^{\max}_{\text{block}}$ are the largest naive and blocked standard errors among the four replicas.

Sampler	(N,D)	E_{mean}	E_{exact}	Acceptance	$SE^{\max}_{\text{naiv}}$	$SE^{\max}_{\text{block}}$	Lag-1 corr.	CPU-s
Importance	(10,3)	24.14216	24.14215	0.997	3×10^{-6}	7×10^{-5}	~ 0.996	33.793
sampling	(50,3)	120.717	120.711	0.997	2×10^{-4}	7×10^{-3}	~ 0.999	285.587
$dt = 0.02$	(100,3)	241.51	241.42	0.997	6×10^{-4}	3×10^{-2}	~ 1	315.081

Table 2: RBM benchmark for the non-interacting anisotropic trap with $\omega_x = \omega_y = 1$ and $\omega_z = 2.82843$, using $N_{\text{hid}} = 4$ hidden nodes. The exact reference energy is $E_{exact} = N(1 + \omega_z/2)$. The columns $SE^{\max}_{\text{naiv}}$ and $SE^{\max}_{\text{block}}$ are the largest naive and blocked standard errors among the four replicas.

effect of changing the learning rate η and the number of hidden nodes N_{hid} during the Adam optimization.

All tested hyperparameter choices converge to energies above the benchmark value. The best result is obtained for $\eta = 0.1$ and $N_{\text{hid}} = 2$, giving the final production result shown in Table 4. Figure 2 shows that the optimization lowers the energy during the first iterations, but the curves remain noisy.

The remaining difference with respect to the benchmark indicates that, although the RBM captures part of the correlation induced by the interaction, the optimized ansatz is not exact for this case. This is expected because the interaction introduces short-range correlations that are more difficult to represent than the non-interacting Gaussian ground state.

Sampler	(N,D)	η	N_{hid}	E
Importance	(2,2)	0.05	2	3.153
sampling		0.05	4	3.158
$dt = 0.02$		0.1	2	3.152
		0.1	4	3.170

Table 3: RBM hyperparameter scan for the interacting isotropic trap with $(N, D) = (2, 2)$. The table compares the final energy obtained after Adam optimization for different learning rates η and numbers of hidden nodes N_{hid}.

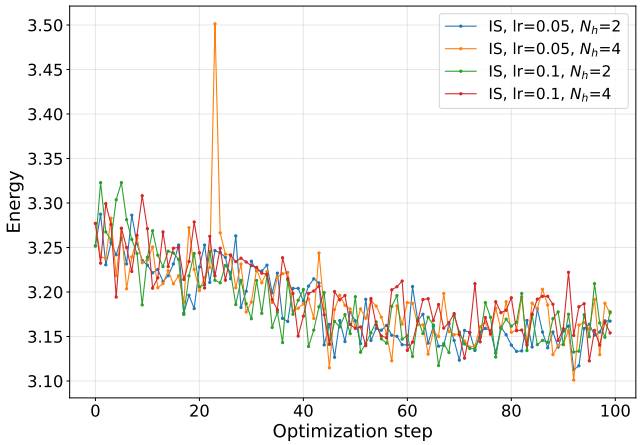

Figure 2: Energy as a function of optimization step for $(N, D) = (2, 2)$ interacting case in isotropic harmonic trap. Comparing the energy optimization for different values of learning rate (lr) and hidden nodes ($N_h \equiv N_{\text{hid}}$).

Interacting anisotropic harmonic trap

Finally, we consider the interacting system in the anisotropic trap. The results are summarized in Table 5, while Figures 3, 4, 5, and 6 show the Adam optimization histories for different learning rates and numbers of hidden nodes.

For $(N, D) = (2, 3)$ and $(N, D) = (10, 3)$, all tested combinations converge to a similar energy around $E \simeq 5.7$ and $E \simeq 58.5$ respectively. The dependence on the learning rate and on N_{hid} is weak once the optimization has reached the plateau.

For $(N, D) = (50, 3)$, the hyperparameters become more important. The runs with larger learning rate and more hidden nodes decrease the energy more efficiently, and the best result is obtained for $\eta = 5 \times 10^{-3}$ and $N_{\text{hid}} = 16$.

The case $(N, D) = (100, 3)$ is considerably more difficult. In contrast to the previous cases, where 100 optimization steps were used, this case employs 130 iterations. However, Figure 6 shows that the optimization is still decreasing after the simulated number of iterations and has not yet reached a clear plateau, indicating that additional optimization steps would be required to achieve convergence. This was not explored further in the present work due to the long run-times associated with this particular simulation.

We therefore do not treat the $N = 100$ interacting result as fully converged. This behavior is consistent

Sampler	(N,D)	E_{mean}	E_{exact}	Acceptance	$SE^{\max}_{\text{naiv}}$	$SE^{\max}_{\text{block}}$	Lag-1 corr.	CPU-s
Importance sampling $dt = 0.02$	(2,2)	3.152	3	0.999	3×10^{-3}	8×10^{-3}	~ 0.456	4.443

Table 4: Final RBM production result for the interacting isotropic benchmark using the best hyperparameters from Table 3. The columns $SE^{\max}_{\text{naiv}}$ and $SE^{\max}_{\text{block}}$ are the largest naive and blocked standard errors among the four replicas.

with the increased number of particle pairs and with the singular structure of the interaction term $1/r_{ij}$: as N grows, close-particle configurations become more likely and the local-energy fluctuations become harder to control with a plain RBM ansatz.

The final production results for the best converged cases are reported in Table 6. The energies are substantially larger than in the corresponding non-interacting anisotropic trap, as expected from the additional repulsive interaction.

Sampler	(N,D)	η	N_{hid}	E_{mean}
Importance sampling $dt = 0.02$	(2,3)	5×10^{-2}	2	5.726
		5×10^{-2}	4	5.729
		1×10^{-1}	2	5.723
		1×10^{-1}	4	5.747
Importance sampling $dt = 0.02$	(10,3)	5×10^{-3}	4	58.513
		5×10^{-3}	8	58.480
		2×10^{-2}	4	58.503
		2×10^{-2}	8	58.566
Importance sampling $dt = 0.02$	(50,3)	1×10^{-3}	8	1238.721
		1×10^{-3}	16	1072.100
		5×10^{-3}	8	784.454
		5×10^{-3}	16	744.376
Importance sampling $dt = 0.02$	(100,3)	5×10^{-4}	16	4758.233
		1×10^{-3}	16	3927.276

Table 5: RBM hyperparameter scan for the interacting anisotropic trap. The table shows the final energy obtained after Adam optimization for different learning rates η and numbers of hidden nodes N_{hid}.

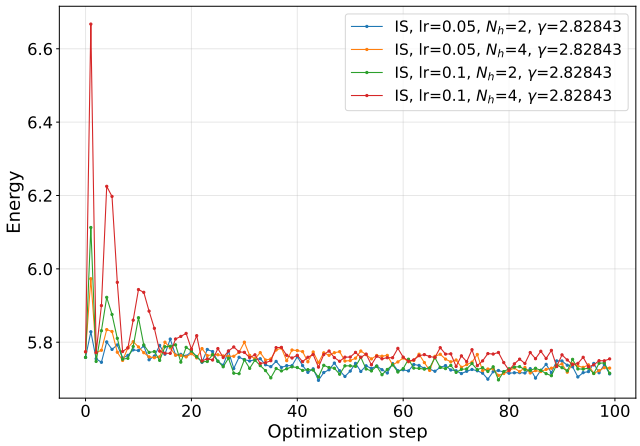

Figure 3: Energy as a function of optimization step for $(N, D) = (2, 3)$ interacting case in anisotropic trap $\omega_z = 2.82843$. Comparing the energy optimization for different values of learning rate (lr) and hidden nodes $(N_h \equiv N_{\text{hid}})$.

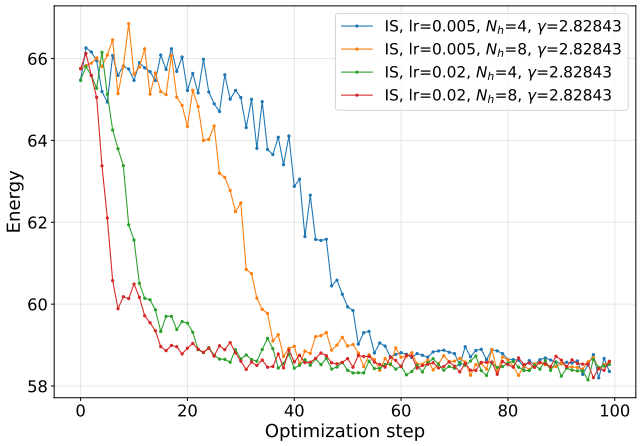

Figure 4: Energy as a function of optimization step for $(N, D) = (10, 3)$ interacting case in anisotropic trap $\omega_z = 2.82843$. Comparing the energy optimization for different values of learning rate (lr) and hidden nodes $(N_h \equiv N_{\text{hid}})$.

Sampler	(N,D)	E_{mean}	η	N_{hid}	Acceptance	$SE^{\max}_{\text{naiv}}$	$SE^{\max}_{\text{block}}$	Lag-1 corr.	CPU-s
Importance	(2,3)	5.723	1×10^{-1}	2	0.997	9×10^{-4}	5×10^{-3}	~ 0.784	9.400
sampling	(10,3)	58.48	5×10^{-3}	8	0.997	4×10^{-3}	6×10^{-2}	~ 0.961	55.407
$dt = 0.02$	(50,3)	744.4	5×10^{-3}	16	0.997	1×10^{-2}	4×10^{-1}	~ 0.993	361.272

Table 6: Final RBM production results for the best converged interacting anisotropic cases with $\omega_x = \omega_y = 1$ and $\omega_z = 2.82843$. The $N = 100$ case is not included because the optimization did not reach a stable plateau. The columns $SE^{\max}_{\text{naiv}}$ and $SE^{\max}_{\text{block}}$ are the largest naive and blocked standard errors among the four replicas.

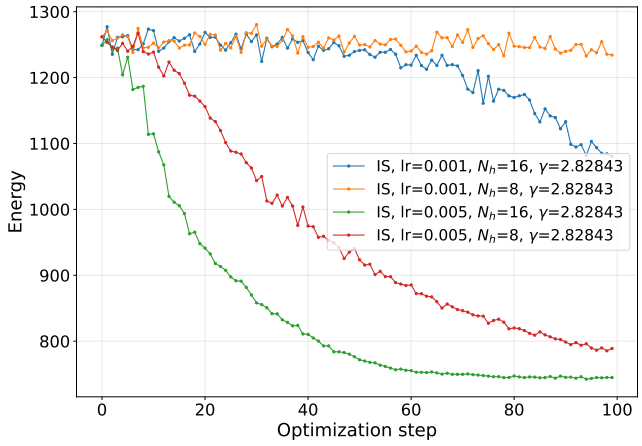

Figure 5: Energy as a function of optimization step for $(N, D) = (50, 3)$ interacting case in anisotropic trap $\omega_z = 2.82843$. Comparing the energy optimization for different values of learning rate (lr) and hidden nodes ($N_h \equiv N_{\text{hid}}$).

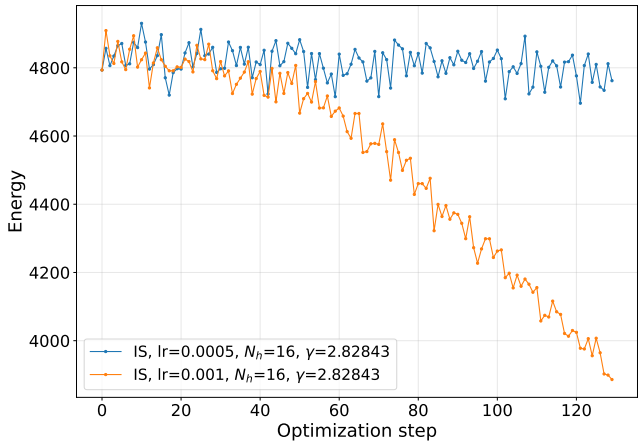

Figure 6: Energy as a function of optimization step for $(N, D) = (100, 3)$ interacting case in anisotropic trap $\omega_z = 2.82843$. Comparing the energy optimization for different values of learning rate (lr) and hidden nodes ($N_h \equiv N_{\text{hid}}$).

Feedforward NN

Non-interacting isotropic 2D harmonic trap

A first test run can be done on a non-interacting bi-dimensional system of $N = 2$ particles. As shown in Figure 7, setting $N_{\text{hid}} = 10$ gives a final energy convergence of about $E = 2.0017 \pm 0.0004$. This value is very close to the analytical $E = 2$ though slightly higher. The pre-training procedure manages to lead the NN to a good agreement with the provided wavefunction φ. The little final overestimation discrepancy $\Delta = +0.0017 \pm 0.0004$ is not only a numerical error, but also an expected behavior of the variational principle: we only expect to find upper-bounds to the ground state energy as long as the NN can only be an approximation to the true ground state wavefunction. In this sense, we can consider Δ to be a lower-bound estimate of the intrinsic approximation error the NN can impose on tests conducted on the following more complex systems.

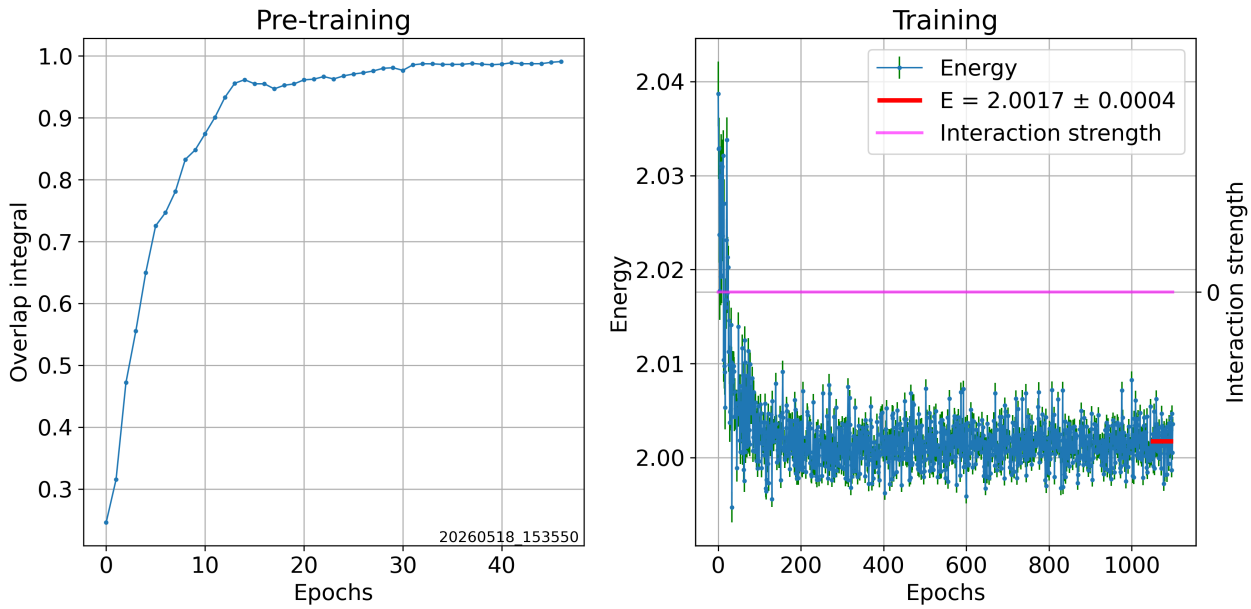

Figure 7: Feedforward NN training for a non-interacting 2D system composed of $N = 2$ particles. No interaction potential is applied. Further details are found in Table 7.

Activation function comparison on an interacting 2D system

We proceed by trying different activation functions $f \in \{\tanh, \text{gelu}, \text{sigmoid}\}$ to find the ground state of an interacting $D = 2$, $N = 2$ system, subject to a Coulombian potential (Eq. 4). We know the analytical energy value is $E = 3$.

As shown in Figures 8 and 9, all three runs produced results within 2% of the expected value. Since the lowest, overall least noisy estimate was obtained using $f = \tanh$ and it was also the activation function of choice in [5], we will employ it throughout the next tests.

Interacting anisotropic harmonic trap

Finally, we move on to $D = 3$ interacting anisotropic systems composed of $N = 3$ and $N = 10$ particles. Dealing with a larger number of particles and introducing anisotropies makes us guess a greater number of hidden nodes could help represent the system's degrees of freedom. Results obtained for $N = 10$ varying the number of hidden nodes $N_{\text{hid}} = 12, 30, 65$ are shown in Figure 10. We see that the best result is given by $N_{\text{hid}} = 30$, whereas higher and lower numbers of hidden nodes seem to struggle more in representing the system faithfully. On the contrary, it is worth noting how the pre-training process seems to benefit greatly from a higher number of hidden nodes. We could guess that running the optimization process with more nodes, different learning rates and many more steps, may lead to better results, but this is only a hypothesis and further investigation is required, going beyond the scope of this project.

Looking at Figure 11, we can see how the radial probability distribution spreads further away from the center of the potential well as the number of particles increases. All results obtained with the feedforward neural network are summarized in Table 7

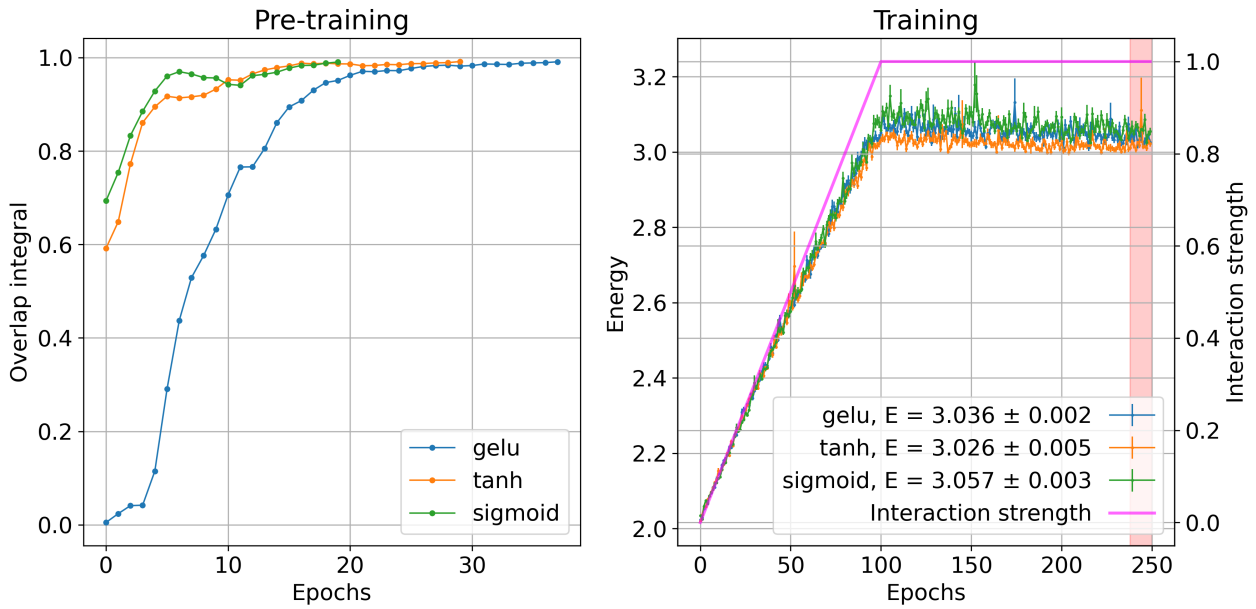

Figure 8: Comparison of training processes using different activation functions in the feedforward neural network. The analyzed 2D system is composed of $N = 2$ particles. A Coulombian interaction potential (Eq. 4) is applied. Further details are found in Table 7. The resulting one-body density plots are shown in Figure 9.

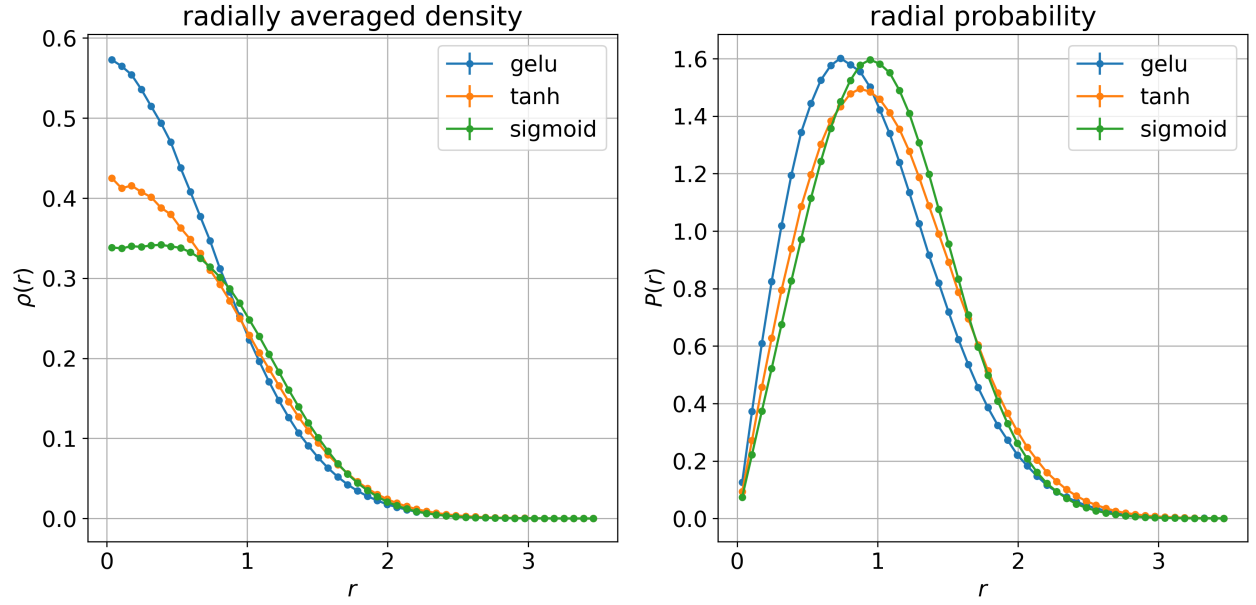

Figure 9: Comparison of one-body density plots resulting from VMC runs using different activation functions in the feedforward NN. The analyzed 2D system is composed of $N = 2$ particles. A Coulombian interaction potential (Eq. 4) is applied. The errorbars shown are indicative underestimates of the true errors, as discussed in Project 1 [3]. Further details are found in Table 7. The training processes which converged to these final wavefunctions are shown in Figure 8.

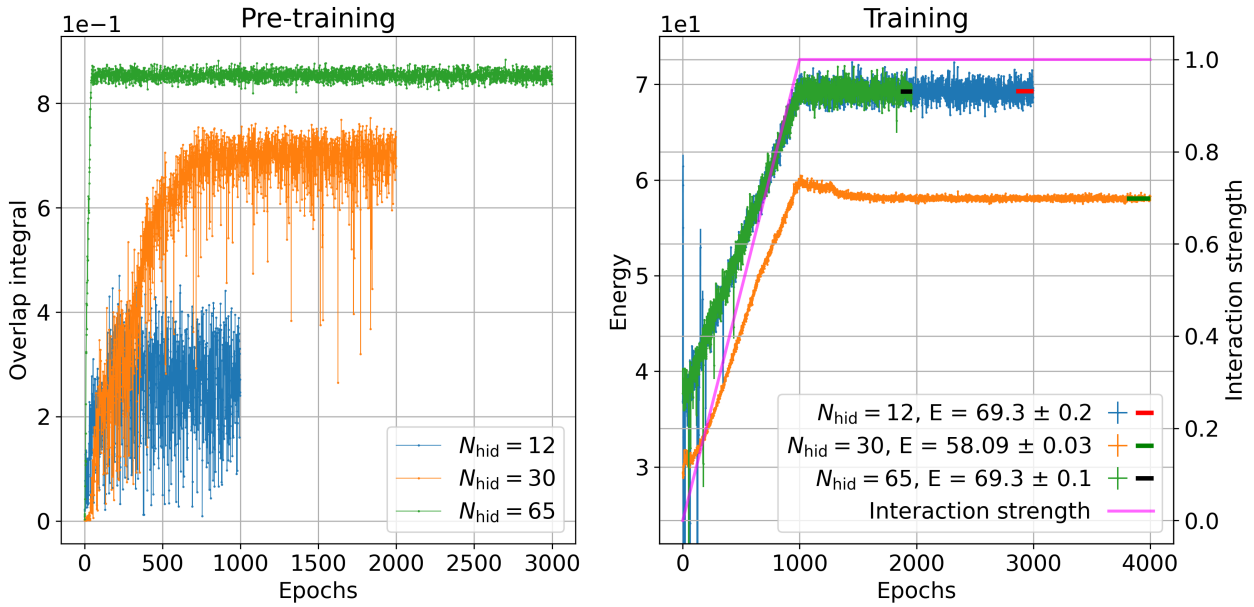

Figure 10: Comparison of training processes using a different number of hidden nodes N_{hid} in the feedforward NN. The analyzed 3D system is composed of $N = 10$ particles; the harmonic trap is anisotropic with $\omega_x = \omega_y = 1$ and $\omega_z = 2.82843$. A Coulombian interaction potential (Eq. 4) is applied. Further details are found in Table 7.

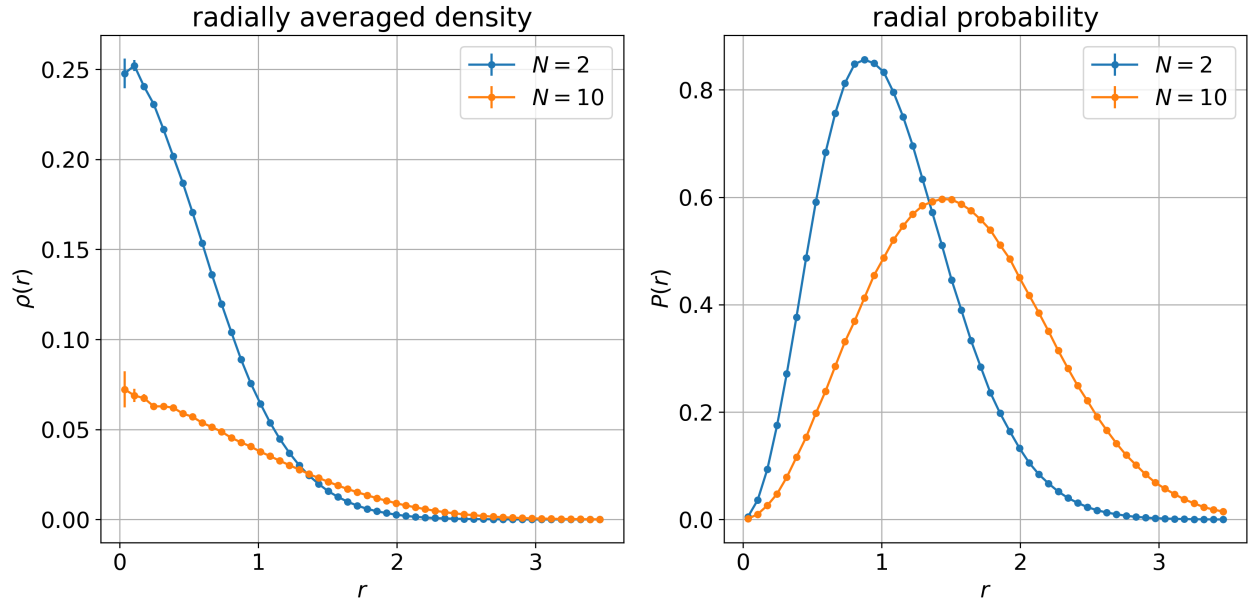

Figure 11: Comparison of one-body density plots resulting from VMC runs employing the feedforward NN. The analyzed 3D systems are subject to the anisotropic harmonic trap with $\omega_x = \omega_y = 1$ and $\omega_z = 2.82843$. A Coulombian interaction potential (Eq. 4) is applied. For the $N = 2$ case, $N_{\text{hid}} = 12$ hidden nodes were used; for the $N = 10$, $N_{\text{hid}} = 12$ instead. Both plots have been scaled for the respective number of particles to facilitate comparison. The errorbars shown are indicative underestimates of the true errors, as discussed in Project 1 [3]. Further details are found in Table 7.

(N,D)	interact.	f	N_{hid}	N_{adiab}	N_{train}	`min_improvement`	`max_patience`	E_{mean}
(2,2)	no	tanh	10	—	1100	0.02	80	2.0017(4)
(2,2)	yes	gelu	10	100	250	—	—	3.036(2)
(2,2)	yes	tanh	10	100	250	—	—	3.026(5)
(2,2)	yes	sigmoid	10	100	250	—	—	3.057(3)
(2,3)	yes	tanh	12	1000	3000	0.02	80	5.688(2)
(3,3)	yes	tanh	12	1000	5118	0.05	80	9.831(5)
(10,3)	yes	tanh	12	1000	3000	0.02	80	69.3(2)
(10,3)	yes	tanh	30	1000	4000	0.02	100	58.09(3)
(10,3)	yes	tanh	65	1000	1961	0.02	200	69.3(1)

Table 7: Detailed summary of VMC results obtained with the feedforward NN. The analyzed 3D systems are subject to the anisotropic harmonic trap with $\omega_x = \omega_y = 1$ and $\omega_z = 2.82843$. All tests were run using 10^4 MC cycles per optimization step; importance sampling was employed with a time step of 0.15, keeping the acceptance rate within the $90 \sim 95\%$ range. The initial learning rate was set to 0.05 and the target value for the overlap integral was set to 0.99.

PINN

Non-interacting isotropic harmonic trap

Table 8 presents the estimated energies of the PINN model for the BEC in the isotropic harmonic trap in the non-interacting case. All the energies estimated with the PINN model, $E_{\mathbf{PINN}}$, are approximately equal to the analytical value $E_{\mathbf{exact}}$. The uncertainty increases with about one order of magnitude when going from 1 to 3 dimensions rising from 10^{-6} to 10^{-5}. The relative error for all dimensions and particle numbers are below 10^{-5}.

The PINN model successfully reproduces the ground state of the isotropic non-interacting systems for particle numbers ranging from $N = 1$ to $N = 10$. This is demonstrated by the agreement between the estimated local energies and the corresponding analytical results. Furthermore, the low variance of the local energy provides additional evidence that the learned wave functions are accurate approximations of the true ground states.

(N, d)	$E_{\mathbf{exact}}$	$E_{\mathbf{PINN}}$
$(1, 1)$	0.5	0.500000000(2)
$(1, 3)$	1.5	1.50000000(2)
$(2, 3)$	3.0	3.00000000(8)
$(10, 3)$	15.0	15.00000000(9)

Table 8: Energies and statistical uncertainties for non-interacting bosons for the isotropic harmonics trap, estimated using the PINN model. The uncertainties are computed using the blocking method.

Energies and variance with different samplers

In Table 9 you can observe the estimate of the local energy, using VMC sampled configurations, for three different models. All models have $N = 10$,$d = 3$, $\omega_x = \omega_y = 1$ and $\omega_z = 2.82843$, and started with $E_\theta = 58.48$. For all three models $\langle E \rangle_{\mathbf{PINN}}$ was lower than the initial E_θ, initialized from RBM results. G2, has the lowest variance $Var(E_{\mathbf{PINN}})$ out of all three models and is also the only model which had a trainable E_θ.

Anisotropic harmonic trap with and without interactions

In table 10 the energy for the non-interacting $a = 0.0$ and interacting case $a = 1.0$ are given for the anisotropic harmonic trap in three dimensions. For $a = 0.0$, we observe that the PINN reproduces the analytical energies with high accuracy. Furthermore, the variance of the local energy is 4.4×10^{-5} for $N = 2$ and 1.27×10^{-4} for $N = 10$. For the interacting case, the PINN model predicts that the energy per particle increases from 2.83 to 5.538, going from 2 to 10 particles. The corresponding variances increase from 1.1×10^{-4} to 0.32, going from 2 to 10 particles.

The PINN model successfully reproduces the ground state of the anisotropic non-interacting systems for particle numbers $N = 2$ and $N = 10$. The low variance of the local energy provides evidence that the learned wave functions are accurate approximations of the true ground states.

For the interacting, anisotropic case, the PINN appears to provide an accurate approximation to the ground-state wave function for $N = 2$, as indicated by the low variance of the local energy, but struggles for $N = 10$. We observe that the variance increases significantly when going from 2 to 10 particles. For $N = 10$, the dimensionality of the configuration space becomes substantially larger, making it harder to sample training data that is a part of the physically relevant regions of configuration space. This may be due to the competition between the Coulomb interactions and the harmonic oscillator potential, which gives rise to a more complex configuration-space structure that is difficult to reproduce using simple samplers. This claim is supported by Figure 11, where we observe that the volume of the bosonic gas increases with the number of particles when interactions are included. Although the ellipsoid sampler was able to generate configurations with values of V_{HO} and V_{C} that more closely matched those obtained from the VMC simulations, it produced poorer PINN results than the

Model name	Trainable E_θ	Initial E_θ	Final E_θ	$E_{\mathbf{PINN}}$	$Var(E_{\mathbf{PINN}})$
E1 (Ellipsoid sampler)	No	58.48	58.48	54.99(4)	6.43
G1 (Gaussian sampler)	No	58.48	58.48	54.96(2)	5.74
G2 (Gaussian sampler)	Yes	58.48	55.89	55.92(2)	**0.32**

Table 9: Comparison of PINN training setups for the $N = 10$, $d = 3$, $\omega_x = \omega_y = 1$ and $\omega_z = 2.82843$ anisotropic interacting system. The table compares the effect of different sampling strategies and the use of either a fixed or trainable energy parameter (E_θ).

Gaussian sampler. This indicates that reproducing configurations that lead to physically realistic values of V_{HO} and V_{C} is not sufficient to ensure that the sampled configurations accurately represent the relevant regions of configuration space. This suggests that more sophisticated sampling methods may be required for PINNs to be a good option for higher particle numbers.

N	a	$E_{\mathbf{PINN}}$	Var($E_{\mathbf{PINN}}$)	$E_{\mathbf{exact}}$
2	0	4.82844(1)	0.000044	4.82843
10	0	24.14216(5)	0.000127	24.1421
2	1.0	5.66999(9)	0.00011	—
10	1.0	55.38(5)	0.32	—

Table 10: Energies and statistical uncertainties for the anisotropic harmonic trap ($\omega_x = \omega_y = 1$ and $\omega_z = 2.82843$) with and without interactions, estimated using the PINN model. The uncertainties are computed using the blocking method.

Comparison of methods

Tables 11 and 12 summarize the main comparison between the methods. For the non-interacting isotropic harmonic trap, Table 11 shows that both RBM and PINN reproduce the analytical reference energies with high accuracy. This confirms that both approaches can represent the ground state of the non-interacting harmonic oscillator. The PINN results are especially close to the exact values in this case.

(N, D)	Exact E_0	RBM	PINN
$(1, 1)$	0.5	0.500002(5)	0.500000000(2)
$(1, 3)$	1.5	1.499999(7)	1.50000000(2)
$(10, 3)$	15.0	14.9998(2)	15.00000000(9)

Table 11: Energy results comparison for the isotropic non-interacting harmonic trap.

The interacting cases are more informative for comparing the flexibility of the different methods. As shown in Table 12, the feedforward NN improves over the RBM for the tested interacting systems, giving lower energies for both the $(N, D) = (2, 2)$ isotropic and $(N, D) = (2, 3)$ and $(10, 3)$ anisotropic traps. The PINN gives the lowest energies in the anisotropic cases where it is available: for $(N, D) = (2, 3)$ it gives $E = 5.66999(9)$, below both the RBM and feedforward NN estimates, and for $(N, D) = (10, 3)$ it gives $E = 55.38(5)$, again below the other two methods. Since, within a variational interpretation, a lower energy corresponds to a better approximation to the true ground state, the PINN gives the best energy estimates among the three approaches for these interacting anisotropic cases.

While the RBM and PINN's results are assigned a blocked error, feedforward NN's results are obtained by averaging the energies found over the last final $\sim 5\%$ of the training steps. Although feedforward NN's results prove to be slightly lower compared to RBM's results (especially as the number of degrees of freedom increases), the time required to train the feedforward NN, which is dependent on a higher number of parameters, is significantly longer (the $(10, 3)$ run took approximately 24 hours).

However, the comparison should be interpreted together with the stability and computational cost of each method. The RBM is the simplest neural-network ansatz considered here and is relatively cheap to optimize, but its plain form has difficulties representing the short-range correlations introduced by the interaction. The feedforward NN has greater flexibility and improves the interacting energies, but it requires a more elaborate pre-training and optimization procedure. The PINN gives the lowest energies, but its performance depends strongly on the quality of the sampled training configurations and on the treatment of the energy parameter in the PDE residual.

The hyperparameter scans also show that increasing the number of hidden nodes does not necessarily improve the result. For the RBM, the effect of increasing N_{hid} is weak for the smaller systems and becomes relevant only for larger interacting systems, such as $(N, D) = (50, 3)$. For the feedforward NN, the same behavior is seen more clearly: in the $(N, D) = (10, 3)$ interacting anisotropic case, the intermediate choice $N_{\text{hid}} = 30$ gives a much better energy than both $N_{\text{hid}} = 12$ and $N_{\text{hid}} = 65$. Thus, a larger network is not automatically better; the result depends on the optimization dynamics, the learning rate, and the ability of the model to converge to a good minimum.

Conclusion

In this project, we studied trapped bosonic systems using three neural-network-based approaches: a Restricted Boltzmann Machine, a feedforward Neural Network, and a Physics-Informed Neural Network. All three methods were tested on non-interacting and interacting harmonic traps, using analytical results or

(N, D)	Trap type	Exact E_0	RBM	Feedforward NN	PINN
$(2, 2)$	isotropic	3	3.152(8)	3.026(5)	–
$(2, 3)$	anisotropic	–	5.723(5)	5.688(2)	5.66999(9)
$(10, 3)$	anisotropic	–	58.48(6)	58.09(3)	55.38(5)

Table 12: Comparison of ground-state energies obtained with the RBM, feedforward NN, and PINN approaches for interacting bosonic systems in harmonic traps. The anisotropic cases correspond to $\omega_x = \omega_y = 1$ and $\omega_z = 2.82843$. Values in parentheses denote the estimated statistical uncertainty in the last quoted digits.

cross-method comparisons as benchmarks.

The RBM implementation successfully reproduced the exact energies of the non-interacting isotropic and anisotropic harmonic traps. This is consistent with previous studies demonstrating that neural-network quantum states can accurately reproduce benchmark many-body energies. In particular, the present results are in agreement with the findings of Carleo and Troyer [1], regarding the ability of neural-network quantum states to accurately represent many-body ground states. The agreement with the analytical ground-state energies further demonstrates that the RBM ansatz, combined with VMC sampling and Adam optimization, accurately captures the Gaussian ground state. For interacting systems, the RBM still produced reasonable variational estimates, but the optimization became more sensitive to the learning rate, the number of hidden nodes, and the system size. In particular, the interacting $N = 100$ anisotropic case did not reach a stable plateau within the simulated optimization time, indicating that the plain RBM ansatz struggles with large interacting systems.

The feedforward NN improved the interacting energies compared to the RBM in the cases where both methods were tested. Its greater flexibility allowed it to obtain lower variational energies, especially for the anisotropic interacting systems. However, this came at the cost of a more complex training procedure, including pre-training, adiabatic introduction of the interaction, and longer optimization times. The results also showed that simply increasing the number of hidden nodes does not guarantee better performance; an intermediate network size gave the best result in the tested $(N, D) = (10, 3)$ case.

The PINN approach reproduced the non-interacting analytical energies with very high accuracy and gave the lowest energies for the interacting anisotropic cases where it was available. This suggests that, among the three approaches studied here, the PINN provided the best approximation to the interacting ground state in terms of energy. Nevertheless, the PINN became more difficult to train for larger interacting systems, as shown by the increased variance for $N = 10$. This indicates that more sophisticated sampling strategies are needed for PINNs to remain reliable at higher particle numbers.

Overall, the three methods show complementary strengths. The RBM is simple and efficient, the feedforward NN is more flexible but more expensive to train, and the PINN gives the most accurate interacting energies in our tests but is strongly dependent on the quality of the training configurations. Future work could improve the comparison by using more systematic hyperparameter searches, longer optimization runs, and more sophisticated sampling methods.

References

[1] G. Carleo and M. Troyer, *Solving the quantum many-body problem with artificial neural networks*, *Science* **355** (2017) 602–606.

[2] T. Macarulla, *Variational monte carlo study of trapped bosons: From the non-interacting harmonic oscillator to hard-sphere interactions*, 2026. Computational Physics II, Project 1 report, University of Oslo.

[3] D. Cacopardi, *Variational monte carlo*, 2026. Computational Physics II, Project 1 report, University of Oslo.

[4] O. Fausko, *Simulations of trapped bose gases with multiple variational monte carlo methods*, 2026. Computational Physics II, Project 1 report, University of Oslo.

[5] H. Saito, *Method to solve quantum few-body problems with artificial neural networks*, *J. Phys. Soc. Jpn.* **87** (2018).

[6] M. Hjorth-Jensen, "Project description." `https://github.com/CompPhysics/ComputationalPhysics2/blob/gh-pages/doc/Projects/2026/Project2/Project2ML/ipynb/Project2ML.ipynb`, 2026.

[7] M. Zaheer, S. Kottur, *et. al.*, *Deep sets*, in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, Curran Associates, Inc. (2017) 3391–3401.

[8] N. D. Drummond, M. D. Towler, and R. J. Needs, *Jastrow correlation factor for atoms, molecules, and solids*, *Physical Review B* **70** (2004) 235119, [`arXiv:0801.0378`].

[9] L. Bertini, M. Mella, D. Bressanini, and G. Morosi, *Explicitly correlated trial wave functions in quantum monte carlo calculations of excited states of be and be^-*, *Journal of Physics B: Atomic, Molecular and Optical Physics* **34** (2001) 257–265.

[10] M. Hjorth-Jensen, "Boltzmann machines, deep learning, and neural quantum states." `https://github.com/CompPhysics/ComputationalPhysics2/blob/gh-pages/doc/pub/week12/ipynb/week12.ipynb`, 2026.